

Praxis: Übergänge zum Kernel beobachten

System Calls und Prozesszustände mit Linux-Werkzeugen sichtbar machen

1.06

Beobachten heißt: Aktion, Übergang, Ergebnis

01 Aktion auslösen

Ein kleines, kontrolliertes Programm öffnet, liest oder schreibt eine Ressource.

02 Übergang erfassen

strace protokolliert System Calls, Signale und Prozesszustandsänderungen.

DREI FRAGEN

Welcher Call?

Welche Argumente?

Welches Ergebnis?

Wir beobachten die Schnittstelle – nicht den gesamten internen Kernelpfad.

Die Laborumgebung bleibt klein und reproduzierbar

VORBEREITUNG

```
$ mkdir -p /tmp/kernel-lab  
$ cd /tmp/kernel-lab  
$ printf 'kernel-lab\n' > demo.txt  
$ command -v strace  
$ uname -sr
```

01 Unkritische Testdaten

Die Übung arbeitet nur unter /tmp und liest eine selbst erzeugte Datei.

02 Normale Benutzerrechte

Root ist für die Grundübungen nicht erforderlich.

Nutze ein Testsystem und keine produktiven oder vertraulichen Prozesse.

Erstes Experiment: cat beim Dateizugriff verfolgen

AUSFÜHREN

```
$ strace -o trace.log \  
-e trace=openat,read,write,close \  
cat demo.txt
```

```
$ sed -n '1,12p' trace.log
```

01 -o trace.log

Die Trace-Ausgabe landet getrennt von der Programmausgabe in einer Datei.

02 -e trace=...

Der Filter reduziert die Ausgabe auf Dateizugriffe und Datenübertragung.

Ein kleiner Filter macht den Übergang schneller lesbar.

Jede Trace-Zeile hat dieselbe Grundstruktur

openat	(AT_FDCWD, "demo.txt", O_RDONLY)	= 3
read	(3, "kernel-lab\n", 131072)	= 11
write	(1, "kernel-lab\n", 11)	= 11
close	(3)	= 0

Name und Argumente stehen links; Rückgabewert oder Fehler steht rechts.

Fehler werden als Rückgabewert sichtbar

FEHLERFALL

```
$ strace -e trace=openat cat fehlt.txt  
openat(AT_FDCWD, "fehlt.txt", O_RDONLY) = -1 ENOENT  
cat: fehlt.txt: No such file or directory
```

01 -1

Der System Call ist fehlgeschlagen.

02 ENOENT

Das symbolische errno erklärt die Fehlerklasse.

Programmmeldung und Kernel-Rückgabe lassen sich direkt miteinander verbinden.

Filter beantworten unterschiedliche Fragen

%file

Welche Pfade und Metadaten nutzt der Prozess?

%desc

Welche File-Descriptor-Operationen treten auf?

%memory

Welche Speicherabbildungen werden verändert?

%process

Welche Calls betreffen Start, Threads und Ende?

Die Statistik zeigt Häufigkeit, Zeit und Fehler

ZUSAMMENFASSUNG

```
$ strace -c cat demo.txt >/dev/null
```

% time	usecs/call	calls	errors	syscall
...	...	...	...	openat
...	...	...	...	read
...	...	...	...	close
100.00		...	...	total

01 -c

Aggregiert Aufrufe, Fehler und gemessene Zeit pro System Call.

02 Interpretieren

Viele Calls bedeuten nicht automatisch viel Laufzeit; Messung enthält Trace-Overhead.

Nutze -c für Muster und Vergleiche – nicht als alleinige Performanceanalyse.

Zeitstempel und Dauer ordnen Ereignisse ein

ZEITLICHE SICHT

```
$ strace -tt -T -o timed.log \  
-e trace=openat,read,write,close \  
cat demo.txt  
  
12:41:03.112 openat(...) = 3 <0.000021>  
12:41:03.113 read(...)   = 11 <0.000015>
```

01 -tt

Schreibt Zeitstempel mit Mikrosekundenauflösung vor die Zeile.

02 -T

Ergänzt die innerhalb des Calls gemessene Dauer.

Zeitangaben helfen bei Reihenfolge und Ausreißern, enthalten aber Beobachtungsaufwand.

Mit -f bleiben Kindprozesse im Trace

PROZESSBAUM

```
$ strace -f -e trace=%process \  
  sh -c 'printf x | wc -c'
```

```
clone(...) = 4210
```

```
[pid 4210] execve(...) = 0
```

```
[pid 4211] execve(...) = 0
```

```
wait4(...) = 4210
```

01 -f

Folgt mit fork, vfork oder clone erzeugten Prozessen und Threads.

02 PID-Markierung

Die Ausgabe ordnet Calls dem jeweiligen Prozess oder Thread zu.

Shell-Pipelines werden erst verständlich, wenn alle beteiligten Prozesse sichtbar sind.

Laufende Prozesse lassen sich kontrolliert anhängen

ATTACH

```
$ sleep 60 &
$ target_pid=$!
$ strace -p "$target_pid"
strace: Process ... attached

# Beenden mit Ctrl+C
# Der Zielprozess läuft anschließend weiter
```

01 -p PID

strace hängt sich an einen vorhandenen Prozess an und löst sich mit Ctrl+C wieder.

02 Berechtigungen

ptrace-Regeln, Benutzergrenzen oder Systemrichtlinien können das Anhängen verhindern.

/proc zeigt Speicherbereiche und File Descriptors

PROZESSSICHT

```
$ pid=$$  
$ sed -n '1,8p' /proc/$pid/maps  
$ ls -l /proc/$pid/fd  
$ cat /proc/$pid/fdinfo/0
```

01 maps

Zeigt virtuelle Bereiche, Rechte, Offsets und zugeordnete Dateien.

02 fd

Symlinks verbinden Descriptor-Nummern mit Dateien, Pipes oder Sockets.

03 fdinfo

Ergänzt descriptor-spezifische Zustandsinformationen.

/proc/<PID>/status ergänzt den Kontext

State

Aktueller Prozesszustand

Threads

Anzahl der Threads

voluntary_ctxt_switches

Freiwillige Kontextwechsel

Seccomp

Aktiver Seccomp-Modus

perf trace ergänzt eine andere Perspektive

OPTIONAL

```
$ perf trace cat demo.txt

0.000 cat/... openat(...) = 3
0.120 cat/... read(...)  = 11
0.180 cat/... write(...) = 11
```

01 Ähnliches Ziel

perf trace zeigt System-Call-Ereignisse in einer perf-basierten Ansicht.

02 Andere Voraussetzungen

Installation, Kernelkonfiguration und Berechtigungen können abweichen.

Nutze zunächst strace; ergänze perf trace, wenn Zeitbezug oder perf-Integration wichtig wird.

Mini-Lab: Beobachtung sauber dokumentieren

01 Hypothese

Notiere vorab, welche Calls und Ressourcen du erwartest.

02 Enger Trace

Starte mit wenigen Calls oder einer passenden Filtergruppe.

03 Belege sichern

Speichere Befehl, Trace, Systemversion und relevante /proc-Ausschnitte.

Trenne Beobachtung und Interpretation: Ein Trace zeigt Übergänge, nicht automatisch deren Ursache.